

# Seat Reservation Service — Design PRD

---

**Date:** 2026-10-04 **Author:** Apeksha Adekar **Status:** Draft for review **Scope:** Backend service + live observability dashboard for Paytm Money take-home exercise

---

## 1. Goals

---

Build, deploy, and operate a JSON HTTP service that sells assigned seats for an event, under heavy concurrent load, with provable correctness and live observability.

### 1.1 Correctness bar (must hold under ~20k concurrent reservations)

1. **No double-sell** — each seat is confirmed to at most one user. For a hot seat with N concurrent attempts: exactly one `201`, N-1 clean `409`.
2. **Zero 5xx** — declines are 4xx domain outcomes, not server errors.
3. **Reconciliation invariant** — `available + held + confirmed == total_seats` holds continuously, during and after the burst.
4. **Idempotency** — same key, exactly one reservation. Same key with different seats → `409`.
5. **Per-user limit** — a user firing 10 parallel reserves against a `limit=4` show ends with at most 4 confirmed seats.
6. **Identity is token-derived** — a request can only act as the token's user and can only cancel its own reservations.

### 1.2 Deploy & Observe bar (equally weighted)

- Public live URL (Fly.io free tier) that survives cold start and comes up healthy.
- Dockerized — clean checkout runs the same way we deploy.
- `/healthz` liveness + `/readyz` readiness (actually checks DB; fails closed when DB is down).
- Prometheus metrics that reconcile with API state and dashboard observations.
- Structured logs with request/correlation id, publicly accessible via `fly logs`.
- One-command burst script ( `./burst.sh <BASE_URL>` ) that reproduces the stampede and prints outcome distribution + final reconciliation.
- Live dashboard at `/dashboard` showing seat map, counters, and recent events in real time via SSE.

## 2. Non-goals

---

- Payment integration — reservations are confirmed instantly, no payment leg.
- Buyer-facing UI — the dashboard is observability-only, not a purchase flow.
- Admin UI — show creation is via `POST /shows` with an admin token; no form.
- Multi-tenant / multi-region — single Fly machine + single MySQL instance.
- Social features, seat recommendations, pricing tiers — out of scope.
- Horizontal scaling — single-machine design; horizontal path noted as future work.

### 3. Decision log

Seven architectural choices, each with its optimized pick and the trade-off we accept.

| # | Decision                 | Chosen                                                                                                                                                                         | Trade-off accepted                                                                                      |
|---|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| 1 | Atomic reserve mechanism | Conditional <code>UPDATE</code> on <code>seats</code> PK + sorted lock order; <code>reservation_seats</code> has filtered <code>UNIQUE (show_id, seat_id)</code> as cheap belt | Double-sell prevention in two places ( <code>UPDATE</code> + unique index), not one                     |
| 2 | Dashboard transport      | SSE from <code>/shows/:id/stream</code> , emit from in-process <code>EventEmitter</code> after <code>COMMIT</code>                                                             | Multi-process scaling would need Redis pub/sub (ponytail: single-machine, add when scaling)             |
| 3 | Partial multi-seat       | All-or-nothing                                                                                                                                                                 | Multi-seat requests have higher reject rate under contention (correct behavior for group bookings)      |
| 4 | Hold model               | Cancel-only, no auto-expiry                                                                                                                                                    | No demonstration of time-boxed hold expiry (exercise allows "choose your model")                        |
| 5 | Confirmation step        | Implicit in <code>POST /reserve</code> (returns <code>status: "confirmed"</code> )                                                                                             | No held→confirm payment leg; abandoned-cart recovery would need a new <code>held</code> state + sweeper |
| 6 | Auth                     | Static bearer tokens, map loaded into memory at startup                                                                                                                        | Cannot add users without restart; mitigated by seeding users with show                                  |
| 7 | Metrics shape            | One counter with <code>outcome</code> label, one gauge per show, one latency histogram                                                                                         | Standard Prom pattern, no trade-off                                                                     |

### 4. API contract

All endpoints return JSON. Money is integer `paise`. All timestamps are ISO-8601 UTC. All non-2xx responses have shape `{ "error": "<code>", "message": "<human>" }`.

#### 4.1 `POST /shows` (admin)

Request:

```
{ "name": "friday-night", "seats": ["A1","A2","A3"], "price_paise": 25000, "per_user_limit": 1 }
```

Headers: `X-Admin-Token: <admin token>` Response 201:

```
{
  "id": "sh_01HV8K...",
  "name": "friday-night",
  "price_paise": 25000,
  "seats": ["A1","A2","A3"],
  "per_user_limit": 1
}
```

```
"per_user_limit": 4,
"seats": [{"seat_id": "A1", "status": "available"}, ...]
}
```

Errors: 401 (missing/wrong admin token), 409 (duplicate name), 400 (empty seats, duplicate seat ids in body, non-positive price).

## 4.2 POST /shows/:id/reserve (user)

Request:

```
{ "seats": ["A12", "A13"], "idempotency_key": "7c3f..." }
```

Headers: Authorization: Bearer <user token> (identity is token-derived; any user\_id in the body is ignored). Response 201 :

```
{
  "reservation_id": "rs_01HV8K...",
  "show_id": "sh_01HV8K...",
  "user_id": "u_42",
  "seats": ["A12", "A13"],
  "amount_paise": 50000,
  "status": "confirmed",
  "created_at": "2026-10-04T10:15:30Z"
}
```

Response 409, error is one of: - seat\_taken — one or more requested seats already held/confirmed by another user. - per\_user\_limit — user already holds  $\geq$  limit seats in this show (or request would exceed). - idempotency\_body\_mismatch — same idempotency\_key seen before with different body. Response 404 — show does not exist. Response 400 — empty seats, duplicate seat ids, missing/invalid idempotency\_key. Response 401 — missing/invalid bearer token.

**Idempotent replay:** same (user\_id, idempotency\_key) + same body → returns the original reservation with the original 201 (or 200 on replay — see §7).

## 4.3 POST /reservations/:id/cancel (owner)

Headers: Authorization: Bearer <user token> Request body: none. Response 200 :

```
{ "reservation_id": "rs_01HV8K...", "status": "cancelled", "cancelled_at": "2026-10-04T10:20" }
```

Errors: 404 (reservation not found), 403 (not the owner), 409 (already cancelled). Seats become available atomically with the cancel.

## 4.4 GET /shows/:id (public)

Response 200 :

```
{
  "id": "sh_01HV8K...",
  "name": "friday-night",

```

```

"price_paise": 25000,
"per_user_limit": 4,
"counts": { "available": 198, "held": 0, "confirmed": 2, "total": 200 },
"seats": [{"seat_id":"A1","status":"confirmed"}, ...]
}

```

Reconciliation invariant: `counts.available + counts.held + counts.confirmed == counts.total`.

## 4.5 Operational endpoints

- `GET /healthz` — liveness. Returns `200 {"ok":true}` as long as the process is up.
- `GET /readyz` — readiness. Pings DB with `SELECT 1`. Returns `200 {"ok":true}` if DB is reachable, `503 {"ok":false,"error":"db_unreachable"}` otherwise.
- `GET /metrics` — Prometheus text format (see §11).
- `GET /shows/:id/stream` — SSE stream, one event per seat state change (see §11).
- `GET /dashboard` — static HTML + JS dashboard (see §17).

## 5. Data model

MySQL 8.0, InnoDB, `utf8mb4`. All PKs are `BIGINT AUTO_INCREMENT` where not stated. All timestamps are `TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3)`.

```

CREATE TABLE users (
  id BIGINT PRIMARY KEY,
  token VARCHAR(128) NOT NULL UNIQUE,
  display_name VARCHAR(64) NOT NULL,
  created_at TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3)
) ENGINE=InnoDB;

CREATE TABLE shows (
  id VARCHAR(32) PRIMARY KEY,                -- ULID
  name VARCHAR(128) NOT NULL UNIQUE,
  price_paise INT UNSIGNED NOT NULL,
  per_user_limit SMALLINT UNSIGNED NOT NULL DEFAULT 4,
  total_seats INT UNSIGNED NOT NULL,
  created_at TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3)
) ENGINE=InnoDB;

CREATE TABLE seats (
  show_id VARCHAR(32) NOT NULL,
  seat_id VARCHAR(16) NOT NULL,
  status ENUM('available','held','confirmed') NOT NULL DEFAULT 'available',
  user_id BIGINT NULL,                        -- current owner if held/confirmed
  reservation_id VARCHAR(32) NULL,           -- current reservation if confirmed
  updated_at TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP,
  PRIMARY KEY (show_id, seat_id),
  INDEX idx_show_user (show_id, user_id),    -- per-user limit check
  FOREIGN KEY (show_id) REFERENCES shows(id)
) ENGINE=InnoDB;

CREATE TABLE reservations (
  id VARCHAR(32) PRIMARY KEY,                -- ULID
  show_id VARCHAR(32) NOT NULL,
  user_id BIGINT NOT NULL,

```

```

idem_key VARCHAR(128) NOT NULL,
body_hash CHAR(64) NOT NULL,          -- sha256 of normalized request body
amount_paise INT UNSIGNED NOT NULL,
status ENUM('confirmed','cancelled') NOT NULL DEFAULT 'confirmed',
created_at TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
cancelled_at TIMESTAMP(3) NULL,
UNIQUE KEY uniq_idem (user_id, idem_key),
INDEX idx_show_user_status (show_id, user_id, status),
FOREIGN KEY (show_id) REFERENCES shows(id)
) ENGINE=InnoDB;

CREATE TABLE reservation_seats (
reservation_id VARCHAR(32) NOT NULL,
show_id VARCHAR(32) NOT NULL,
seat_id VARCHAR(16) NOT NULL,
active_key VARCHAR(64) GENERATED ALWAYS AS
  (CASE WHEN cancelled_at IS NULL THEN CONCAT(show_id,':',seat_id) ELSE NULL END) VIRTUAL,
cancelled_at TIMESTAMP(3) NULL,
PRIMARY KEY (reservation_id, seat_id),
UNIQUE KEY uniq_active_seat (active_key), -- schema-level double-sell backstop
FOREIGN KEY (reservation_id) REFERENCES reservations(id)
) ENGINE=InnoDB;

```

Note: `uniq_active_seat` is a NULL-tolerant unique on a generated column — multiple cancelled rows can coexist (`NULL ≠ NULL`), but at most one active row per `(show_id, seat_id)` can exist. This is the belt. The suspenders is the conditional `UPDATE` on `seats` (see §6).

## 6. Atomic reserve algorithm

Pseudocode. Runs inside a single MySQL transaction with `tx_isolation = READ-COMMITTED` (default; sufficient because all our writes use row-level X-locks via `UPDATE`).

```

reserve(user_id, show_id, seat_ids[], idem_key, body_hash):
  BEGIN TX

  # Step 1 – idempotency fast path (before any locks)
  r = SELECT * FROM reservations WHERE user_id=? AND idem_key=?
  if r exists:
    if r.body_hash == body_hash:
      ROLLBACK; return 200, original reservation payload  # idempotent replay
    else:
      ROLLBACK; return 409 idempotency_body_mismatch

  # Step 2 – sort seat_ids ascending (deterministic lock order → no deadlock)
  sorted_seats = sort(seat_ids)

  # Step 3 – per-user limit check (held row lock on all current confirmed rows for this user)
  current_count = SELECT COUNT(*) FROM reservation_seats rs
    JOIN reservations r ON rs.reservation_id = r.id
    WHERE r.show_id=? AND r.user_id=? AND r.status='confirmed'
  FOR UPDATE
  if current_count + len(sorted_seats) > show.per_user_limit:
    ROLLBACK; return 409 per_user_limit

  # Step 4 – atomic seat acquisition (all-or-nothing)
  reservation_id = ulid()

```

```

for seat_id in sorted_seats:
    affected = UPDATE seats
        SET status='confirmed', user_id=?, reservation_id=?
        WHERE show_id=? AND seat_id=? AND status='available'
    if affected != 1:
        ROLLBACK; return 409 seat_taken

# Step 5 – persist the reservation record
try:
    INSERT INTO reservations (id, show_id, user_id, idem_key, body_hash, amount_paise, status)
    VALUES (reservation_id, ?, ?, ?, ?, len(seats)*price, 'confirmed')
catch unique violation on uniq_idem:
    # Concurrent request with same (user_id, idem_key) won the race
    ROLLBACK
    r = SELECT * FROM reservations WHERE user_id=? AND idem_key=?
    if r.body_hash == body_hash: return 200, r # replay
    else: return 409 idempotency_body_mismatch

# Step 6 – persist seat links (unique on active_key is the schema backstop)
try:
    INSERT INTO reservation_seats (reservation_id, show_id, seat_id) VALUES ...
catch unique violation on uniq_active_seat:
    # This should be impossible if Step 4 was correct – the belt caught a suspenders failure
    ROLLBACK
    log.error("double_sell_backstop_tripped", ...)
    return 409 seat_taken

COMMIT
emit_sse_events(show_id, sorted_seats, 'confirmed', user_id) # outside TX
return 201, reservation_payload

```

## 6.1 Why this is race-free

- **Single-seat contention (500 users on A12):** Step 4 issues `UPDATE ... WHERE status='available'`. InnoDB takes a row-level X-lock on `(show_id='X', seat_id='A12')` via the PK. Lock requests serialize. The first transaction's UPDATE sets `status='confirmed'` and commits — all subsequent transactions in the queue find `status='confirmed'`, their WHERE clause matches 0 rows, `affected=0`, they roll back cleanly with `409 seat_taken`. No two UPDATES can both see `status='available'` for the same seat.
- **Multi-seat deadlock avoidance:** Step 2 sorts seats before locking. Two concurrent requests for overlapping seat sets will attempt to acquire row locks in the same global order → no cycle → no deadlock. Lock-wait timeout can still fire under extreme contention; we handle it as `409 seat_taken` (not 5xx).
- **Per-user limit under contention:** Step 3's `SELECT ... FOR UPDATE` takes shared-exclusive locks on all this user's current reservation rows for this show. Two concurrent reserves from the same user serialize on these locks → both see the correct current count. Combined with Step 4's atomic seat flip, a user cannot exceed `per_user_limit`.
- **Belt + suspenders backstop:** If Step 4 had a bug and let two users both pass the check for the same seat, Step 6's `INSERT INTO reservation_seats` would fail on `uniq_active_seat` because `active_key = 'show:seat'` already exists. The second request rolls back with 409. We log at error — this is a "should never happen" invariant.

## 6.2 Lock-wait timeout tuning

- `innodb_lock_wait_timeout = 2` (seconds). Fail fast under extreme contention; the burst harness will see `409 seat_taken` rather than long tail latency.
- Application catches `ER_LOCK_WAIT_TIMEOUT` as a 409 outcome ( `seat_taken` ), not 5xx.

## 7. Idempotency contract

---

- **Scope:** per ( `user_id`, `idem_key` ) . Two different users with the same string are independent.
- **Storage:** `reservations.idem_key + UNIQUE (user_id, idem_key)` . Also `reservations.body_hash = sha256(normalize(request_body))` .
- **Normalization:** `sort(seats) + strip whitespace from idem_key` before hashing. Prevents false mismatches from re-ordered JSON.
- **Exactly-once:** enforced by the unique constraint. Two concurrent reserves with the same key: one INSERTs successfully, the other catches the unique violation and returns the first one's payload.
- **Replay:** same key + same body hash → `200 OK` with the original reservation payload (not `201` ; the second response reports the first write).
- **Mismatch:** same key + different body hash → `409 idempotency_body_mismatch` .
- **TTL:** none for this exercise. In production we'd expire idempotency records after N days.

## 8. Cancellation

---

- `POST /reservations/:id/cancel` : 1. `BEGIN TX` 2. `SELECT * FROM reservations WHERE id=? FOR UPDATE` 3. If not found → `ROLLBACK; 404` 4. If `r.user_id != token.user_id` → `ROLLBACK; 403` 5. If `r.status == 'cancelled'` → `ROLLBACK; 409 already_cancelled` 6. `UPDATE reservations SET status='cancelled', cancelled_at=NOW(3) WHERE id=?` 7. `UPDATE reservation_seats SET cancelled_at=NOW(3) WHERE reservation_id=?` (releases `active_key` → `NULL`) 8. `UPDATE seats SET status='available', user_id=NULL, reservation_id=NULL WHERE reservation_id=?` 9. `COMMIT` , emit SSE events for released seats
- A released seat is immediately re-bookable by the next reserver because `seats.status='available'` .
- A cancel can never "resurrect" another user's confirmation: step 4 enforces ownership; step 8 scopes to `reservation_id=?` (not `seat_id` ).

## 9. Authentication

---

- `users` table seeded by the admin at startup (or by `POST /shows` optionally seeding N test users).
- At boot, service loads `SELECT id, token FROM users` into an in-memory `Map<token, user_id>` .
- Middleware: `Authorization: Bearer <token>` → look up in map → set `req.user_id` . Missing or unknown token → `401` .
- `X-Admin-Token` compared against env `ADMIN_TOKEN` for `POST /shows` . Timing-safe compare.
- Trade-off: adding users requires restart. Mitigation: seed script writes N users at deploy time; the burst harness reads their tokens from a seed JSON.

## 10. Observability

---

### 10.1 Metrics (Prometheus text, `/metrics`)

- `reservations_total{outcome, show_id}` — counter. `outcome` ∈ {confirmed, seat\_taken, per\_user\_limit, idempotent\_replay, idempotency\_body\_mismatch, validation\_error}.
- `cancellations_total{show_id}` — counter.
- `seats_available{show_id}` — gauge, updated on every seat state change.
- `seats_confirmed{show_id}` — gauge.
- `reserve_latency_seconds{show_id}` — histogram, buckets [0.005, 0.01, 0.025, 0.05, 0.1, 0.25, 0.5, 1, 2, 5].
- `http_requests_total{method, route, status}` — counter.
- `db_pool_connections{state}` — gauge, `state` ∈ {active, idle}.
- `sse_subscribers{show_id}` — gauge.

The gauges MUST reconcile with API state: `GET /shows/:id` returns counts that equal `seats_available + seats_held + seats_confirmed`.

### 10.2 Structured logs

- JSON per line via `pino` (or similar). Fields: `ts`, `level`, `req_id`, `user_id` (if known), `show_id` (if known), `route`, `status`, `latency_ms`, `outcome`, `error_code`.
- `req_id` generated per request (ULID). Returned in `X-Request-Id` response header. Included in every log line produced during that request.
- No PII logged.

### 10.3 Live dashboard (SSE)

- `GET /shows/:id/stream` opens an SSE connection.
- Event types:
  - `baseline` — one event on connect with full seat map (same shape as `GET /shows/:id`).
  - `seat` — on every seat state change: {`seat_id`, `status`, `user_id`, `at`}.
  - `counts` — periodic (every 500ms if changed): {`available`, `held`, `confirmed`}.
  - `reservation` — on each confirmed/cancelled reservation: {`reservation_id`, `user_id`, `seats[]`, `outcome`, `at`}.
- Backpressure: in-process `EventEmitter` + per-subscriber bounded queue (drop oldest if a client falls more than 1000 events behind; emit `drop` event).

## 11. Deployment

---

### 11.1 Platform

- **Fly.io** single machine, `shared-cpu-1x` 256MB (free tier).
- **MySQL 8.0** running on the same machine on a Fly persistent volume (free tier allows 3GB). Mounted at `/data/mysql`.
- Alternative: external MySQL (PlanetScale free tier) — rejected to keep deploy self-contained for the interviewer.

## 11.2 Dockerfile (one process, MySQL as sidecar via supervisord)

- Base: `node:22-alpine`.
- Install MySQL 8 server, `supervisord`, create unprivileged `app` user.
- Multi-stage: builder stage runs `pnpm install && pnpm build`; final stage copies `/dist` + `node_modules` + MySQL binaries.
- Entrypoint runs `init-db.sh` (idempotent: creates DB + schema + seeds users if volume is fresh), then `supervisord -n`.
- Supervisord manages `mysqld` + `node /dist/server.js`.
- Health probe hits `/readyz`.

## 11.3 Local dev (docker-compose)

- Two services: `app` (Node + mounted source) + `mysql` (bind-mounted to `./data/mysql`).
- MySQL exposed on host port **3307** to avoid colliding with any existing MySQL on 3306.
- `.env.example` → copy to `.env` → `docker-compose up` → `curl http://localhost:8080/healthz`.

## 11.4 CI (optional, GitHub Actions)

- On push: `pnpm install`, `pnpm test`, `pnpm build`, `docker build`.
- No deploy automation — `fly deploy` manual per commit (keeps the exercise auditable).

# 12. Burst harness

---

## 12.1 `burst.sh <BASE_URL>` behavior

Node script (or Bash wrapper around a Node script) that: 1. Reads seed JSON ( `seed/burst.json` ) → list of `{user_id, token}` × N (default 500). 2. Reads scenarios: `hot_seat` (all N users target seat A12), `random` (each user picks 1 random seat), `multi_seat` (each user picks 2 adjacent seats). 3. For each scenario: fires N requests concurrently (via `Promise.all` with HTTP keep-alive agent, or `undici` pool). 4. Collects per-request outcome: HTTP status + `error` code if 4xx. 5. Prints a report:

```
=== burst.sh report ===
show: friday-night (sh_01HV8K...)
total requests fired: 20000
scenarios: hot_seat (500), random (10000), multi_seat (9500)

outcomes:
  201 confirmed                ... 198
  409 seat_taken                ... 19700
  409 per_user_limit            ... 72
  409 idempotency_body...       ... 0
  200 idempotent_replay         ... 30
  5xx server_error              ... 0 ← must be 0

reconciliation (post-burst GET /shows/sh_01HV8K...):
  available=2, held=0, confirmed=198, total=200 ✓
  confirmed count matches 201 responses: ✓
```

## 12.2 Flags

- `--users N` (default 500)
- `--duration S` (default 10s — spreads requests over window)
- `--instant` (fires all N requests at t=0 instead of over window; the "stampede" mode)
- `--scenarios hot_seat, random, multi_seat` (default: all three)
- `--seat A12` (which seat is "hot")
- `--retries 2` (how many retries per request with same idempotency key — proves idempotency under retry)

## 13. Testing strategy

| Level          | Scope                                                                                                | Tool                                           |
|----------------|------------------------------------------------------------------------------------------------------|------------------------------------------------|
| Unit           | Pure functions (idem key normalization, request validation, seat sort, body hashing)                 | Vitest                                         |
| Integration    | Each endpoint against real MySQL (docker-compose)                                                    | Vitest + testcontainers OR local MySQL on 3307 |
| Contention     | Spawn 100 concurrent reserves on 1 seat; assert exactly $1 \times 201 + 99 \times 409$               | Vitest + Promise.all                           |
| Idempotency    | Fire same key 10x in parallel; assert 1 reservation; fire same key with different body; assert 409   | Vitest                                         |
| Burst          | <code>burst.sh</code> against local, then against Fly                                                | Script                                         |
| Reconciliation | After each test, <code>GET /shows/:id</code> and assert <code>available+held+confirmed==total</code> | Vitest assertion helper                        |
| Health         | <code>/readyz</code> fails when DB container is paused                                               | docker CLI in test                             |

TDD: each endpoint starts with a failing integration test → minimum implementation → passing test → commit.

## 14. Risks & mitigations

| Risk                                                                  | Likelihood | Impact                       | Mitigation                                                                                                                                          |
|-----------------------------------------------------------------------|------------|------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| MySQL on same machine runs out of memory under burst (256MB Fly tier) | Medium     | Service dies mid-burst → 5xx | Tune <code>innodb_buffer_pool_size=64M</code> , cap Node heap <code>--max-old-space-size=128</code> ; test burst locally first with same mem limits |
| Lock-wait timeouts under extreme contention classified as 5xx         | Medium     | Violates "zero 5xx" bar      | Catch <code>ER_LOCK_WAIT_TIMEOUT</code> → map to 409 <code>seat_taken</code>                                                                        |
| Fly free-tier cold start takes >10s                                   | Low        | Burst fails at t=0           | Pre-warm: send <code>/readyz</code> ping before burst starts; wait for <code>200</code>                                                             |

| Risk                                                                 | Likelihood | Impact                              | Mitigation                                                                                               |
|----------------------------------------------------------------------|------------|-------------------------------------|----------------------------------------------------------------------------------------------------------|
| In-process SSE events lost if client reconnects                      | Medium     | Dashboard shows stale state briefly | Dashboard re-GETs baseline on reconnect; SSE auto-reconnects                                             |
| Supervisord restarts crashed <code>mysqld</code> → volume corruption | Low        | Data loss                           | Test by <code>kill -9 mysqld</code> in container; InnoDB recovery handles it                             |
| Admin token leaks via logs                                           | Low        | Anyone can create shows             | Never log request bodies; redact <code>Authorization</code> and <code>X-Admin-Token</code> in middleware |

## 15. Repo layout

```

├── Dockerfile
├── docker-compose.yml
├── fly.toml
├── package.json
├── pnpm-lock.yaml
├── tsconfig.json
├── .env.example
├── README.md
├── WRITEUP.md                                # the required design writeup for the interviewer
├── docs/
│   ├── superpowers/
│   │   ├── specs/
│   │   │   ├── 2026-10-04-seat-reservation-prd.md        (this doc)
│   │   │   └── 2026-10-04-frontend-dashboard-design.md  (dashboard design)
│   │   └── plans/
│   │       └── 2026-10-04-seat-reservation-plan.md        (impl plan, next step)
├── seed/
│   ├── users.json          # 500 pre-seeded user tokens for burst
│   └── show.json           # sample show config
├── sql/
│   └── 001_init.sql        # DDL from §5
├── scripts/
│   ├── init-db.sh
│   ├── seed.ts
│   └── burst.ts            # the one-command burst harness
├── burst.sh                # thin wrapper: node scripts/burst.ts $@
├── public/
│   └── dashboard.html      # static dashboard (see §17)
├── src/
│   ├── server.ts           # app bootstrap
│   ├── config.ts
│   ├── db.ts              # mysql2 pool
│   ├── logger.ts          # pino + req_id
│   ├── metrics.ts         # prom-client registry
│   ├── auth.ts            # token→user_id map
│   ├── events.ts          # in-process EventEmitter for SSE
│   └── routes/
│       ├── shows.ts        # POST /shows, GET /shows/:id
│       ├── reserve.ts      # POST /shows/:id/reserve
│       ├── cancel.ts       # POST /reservations/:id/cancel
│       └── stream.ts       # GET /shows/:id/stream (SSE)

```

|                         |                                                        |
|-------------------------|--------------------------------------------------------|
| ├── dashboard.ts        | # GET /dashboard                                       |
| ├── ops.ts              | # /healthz /readyz /metrics                            |
| ├── domain/             |                                                        |
| │   ├── reserve.ts      | # atomic reserve algorithm (\$6)                       |
| │   ├── idempotency.ts  | # normalize + hash                                     |
| │   ├── errors.ts       | # domain error types → HTTP mapping                    |
| ├── middleware/         |                                                        |
| │   ├── requestId.ts    |                                                        |
| │   ├── errorHandler.ts | # catches thrown errors → 4xx (never 5xx for domain on |
| │   └── access.ts       | # auth + admin check                                   |
| └── test/               |                                                        |
| ├── unit/               |                                                        |
| ├── integration/        |                                                        |
| └── contention/         |                                                        |

## 16. Build order

Fourteen tasks, TDD per task, bite-sized commits. Full plan is produced by `superpowers:writing-plans` after this spec is approved. Headline order:

1. Scaffold + lint + `.env` + Dockerfile + compose.
2. DB schema + migrations + seed.
3. Logger + metrics + `req_id` middleware.
4. Auth middleware + user seed loader.
5. `POST /shows` + `GET /shows/:id`.
6. `POST /reserve` single-seat (basic path).
7. Multi-seat with sorted lock order.
8. Per-user limit.
9. Idempotency (key + body hash + unique constraint).
10. `POST /cancel`.
11. SSE stream + in-process event bus.
12. Dashboard HTML + JS.
13. `/healthz` + `/readyz` + `/metrics` wiring.
14. Burst harness + Fly deploy + README + WRITEUP.

## 17. Frontend dashboard design (summary; full doc at `2026-10-04-frontend-dashboard-design.md`)

- **Purpose:** show the interviewer that the service is behaving correctly during their own burst — seat state updating in real time, hot-seat contention visible, counters reconciling.
- **Stack:** single static HTML file + vanilla JS + `EventSource`. No build step, no framework. Served by the Node app at `/dashboard`. **~150 lines.**
- **Layout:** header (show name, counters), seat grid (color-coded by state), recent reservations feed (last 20), connection status badge.
- **Color language:** available = neutral gray, held = amber, confirmed = red, hot-seat flash = brief pulse on state change. Also encode state in a text label (not color-only) for a11y.
- **States handled:** connecting / live / disconnected (auto-reconnect) / show not found.
- **Non-goals:** no buyer flow, no auth UI, no admin forms (curl handles those).

## Approval

---

This spec is ready for review. Once approved, next step is `superpowers:writing-plans` to produce the implementation plan corresponding to the build order in §16.

# Live Dashboard — Frontend Design

**Date:** 2026-10-04 **Author:** Apeksha Adekar **Status:** Draft for review **Parent spec:** 2026-10-04-seat-reservation-prd.md

## 1. Purpose

A single page served at `GET /dashboard` that lets the interviewer **watch the service behave correctly during their own burst**. It is not a buyer flow or admin UI — those stay as `curl`. The dashboard's only job: make the correctness invariants visible in real time.

## 2. Non-goals

- No buying. No seat picker, no cart, no checkout.
- No login. The dashboard is read-only and public.
- No admin forms. `POST /shows` stays on `curl` so the interviewer can see the raw request.
- No build step. No React, no TypeScript at the frontend, no bundler. Just HTML + CSS + vanilla JS.
- No historical data. Dashboard shows "now". A refresh = new baseline.

### 3. Stack

- **One file:** `public/dashboard.html` served statically by the Node app.
- **Vanilla JS**, ES modules, no transpile.
- **EventSource** for the SSE connection (built into every browser).
- **Minimal CSS**, scoped to the file. CSS grid for the seat map.
- **No deps.** Target: under 200 lines total (HTML + CSS + JS).

ponytail: single-file vanilla; add bundler + framework when the UI grows beyond one page.

## 4. Layout

|                                |                                           |                              |
|--------------------------------|-------------------------------------------|------------------------------|
| friday-night                   | [● live]                                  | ← header                     |
| available 198                  | held 0 confirmed 2 total 200 ✓ reconciled |                              |
| A1 A2 A3 A4 A5 A6 A7 A8 A9 A10 |                                           | ← seat grid<br>(color coded) |
| B1 B2 B3 B4 B5 B6 B7 B8 B9 B10 |                                           |                              |
| C1 C2 C3 C4 C5 C6 C7 C8 C9 C10 |                                           |                              |
| ...                            |                                           |                              |
| recent (last 20)               |                                           | ← event feed                 |

|              |            |          |                    |
|--------------|------------|----------|--------------------|
| 10:15:30.142 | confirmed  | A12      | by u_42            |
| 10:15:30.138 | confirmed  | B3,B4,B5 | by u_17            |
| 10:15:29.901 | cancelled  | A1       | by u_8             |
| 10:15:29.880 | seat_taken | A12      | by u_99 (declined) |
| ...          |            |          |                    |

Fixed layout. Header stays visible; seat grid scrolls; event feed scrolls independently. No modals, no tabs, no collapse. Everything on one screen.

## 5. Visual language

### 5.1 Seat colors

| State        | Fill                    | Border  | Text label |
|--------------|-------------------------|---------|------------|
| available    | #e5e7eb (neutral 200)   | #9ca3af | —          |
| held         | #fde68a (amber 200)     | #f59e0b | H          |
| confirmed    | #fecaca (red 200)       | #dc2626 | C          |
| just-changed | pulse animation (300ms) | —       | —          |

**Important:** state is also encoded in the text label ( H , C , blank), not just color. This is for a11y — colorblind users and screen readers can still read the state. The seat id ( A12 ) is always visible.

### 5.2 Hot-seat storm indicator

When the service rejects  $\geq 10$  requests for the same seat within 2 seconds, the seat gets a brief red glow ( box-shadow pulse, 500ms). This visualizes the hot-seat contention the exercise explicitly tests.

### 5.3 Reconciliation badge

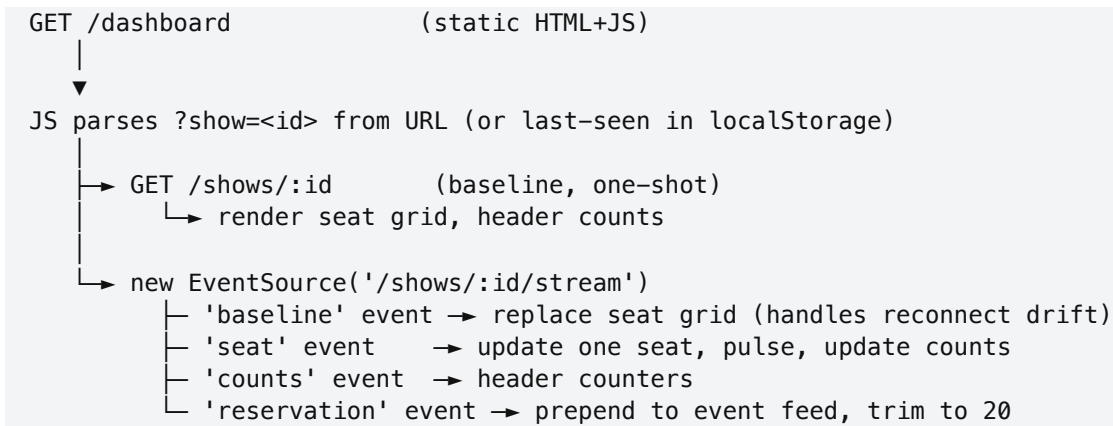
Top header shows ✓ reconciled (green) or ✗ drift: counts don't match seat states (red). Computed client-side from the SSE counts event + the running seat state. If this ever shows red, there's a bug — this IS the invariant check for the viewer.

### 5.4 Connection status

Small badge next to the show name: - ● live (green) — SSE connected. - ● reconnecting (amber) — connection dropped, EventSource auto-retrying. - ● offline (red) — repeated failures (>3 reconnects).

## 6. Data flow

Browser load  
↓  
▼



Reconnection: `EventSource` handles exponential backoff automatically. On reconnect, we discard local seat state and wait for the next `baseline` event — this guarantees we never show stale state after a disconnect.

## 7. States

| Dashboard state  | Trigger                  | What user sees                               |
|------------------|--------------------------|----------------------------------------------|
| loading          | page just opened         | spinner, "connecting to show ..."            |
| live             | baseline + SSE connected | full grid, live updates                      |
| reconnecting     | SSE error event          | amber badge, grid frozen at last known state |
| show not found   | GET /shows/:id → 404     | centered message with input to pick a show   |
| no show selected | URL has no ?show=        | input: show id or name + "load" button       |

## 8. Accessibility

- Minimum 4.5:1 contrast for all text on fills (checked at design time).
- State encoded by both color AND text label (never color-only).
- Seat grid is a `<table>` with `<caption>` naming the show; each seat `<td>` has `aria-label="seat A12, confirmed"`.
- SSE events announced via `aria-live="polite"` region (opt-in via a toggle; defaults off because it's noisy during a burst).
- Keyboard: `?show=` input is tab-focusable; grid is read-only so no interactive focus needed.
- No animations when `prefers-reduced-motion` is set — pulse becomes a static color swap.

## 9. HTML skeleton (indicative, not final)

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
```

```

<title>Seat Reservation – Live Dashboard</title>
<style>/* ~60 lines of scoped CSS */</style>
</head>
<body>
  <header>
    <h1 id="show-name"></h1>
    <span id="conn-status" class="status loading">connecting...</span>
    <div id="counts">available <b id="c-available"></b> · held <b id="c-held"></b> · con
  </header>
  <main>
    <section id="seat-grid-wrap">
      <table id="seat-grid" aria-live="off"><caption id="grid-caption">seats</caption></table>
    </section>
    <aside id="feed">
      <h2>recent</h2>
      <ol id="feed-list"></ol>
    </aside>
  </main>
  <script type="module">
    // ~80 lines: parse URL, fetch baseline, open EventSource, dispatch events to DOM update
  </script>
</body>
</html>

```

## 10. SSE event schema (contract with backend §10.3)

```

// baseline – sent once on connect and on each reconnect
{ "type": "baseline", "counts": { "available": 198, "held": 0, "confirmed": 2, "total": 200 },
  "seats": [ { "seat_id": "A1", "status": "available" }, ... ] }

// seat – one seat changed state
{ "type": "seat", "seat_id": "A12", "status": "confirmed", "user_id": "u_42", "at": "2026-10-04T10:15:30.142Z" }

// counts – periodic, only if changed since last emit
{ "type": "counts", "available": 197, "held": 0, "confirmed": 3, "total": 200 }

// reservation – a reservation was created or cancelled
{ "type": "reservation", "reservation_id": "rs_01HV8K...", "user_id": "u_42",
  "seats": [ "A12" ], "outcome": "confirmed", "at": "2026-10-04T10:15:30.142Z" }

```

The backend MUST send `baseline` within 500ms of connect; dashboard enters `live` state only after receiving it.

## 11. Testing

Manual script, included in `README.md` :

1. Open `/dashboard?show=<id>` in two browser tabs.
2. From a third tab (or `curl`), reserve a seat.
3. Both dashboards flash that seat → red within 200ms.
4. Cancel the reservation.
5. Both dashboards flash the seat → gray.

6. Run `./burst.sh` and watch the grid repaint in real time; verify reconciliation badge stays green throughout.
7. Pause the Node container ( `docker pause` ), see badge go amber then red; unpause, see baseline arrive.

No automated frontend tests. The dashboard is observability-only; its correctness is verified by eye against the API state, which has its own test suite.

## 12. What this intentionally skips

---

| Skipped                                              | When to add                                                            |
|------------------------------------------------------|------------------------------------------------------------------------|
| Charts / graphs over time                            | If we need to prove latency or throughput visually (Grafana is better) |
| Multi-show selector / nav                            | If we have more than one show in demo                                  |
| Dark mode                                            | If someone is actually using this at night                             |
| Export / snapshot button                             | If an interviewer asks "can I save this view"                          |
| Authenticated per-user view ("show my reservations") | Changes dashboard from observability tool to buyer tool — scope creep  |
| Build pipeline (Vite, bundler)                       | When the single file exceeds ~300 lines                                |

---

## Approval

---

This dashboard is deliberately small. One file, no deps, under 200 lines, serves the exercise's "Deploy & Observe" need directly. Review and either approve or push back on specific items; after approval it becomes task #12 in the implementation plan.